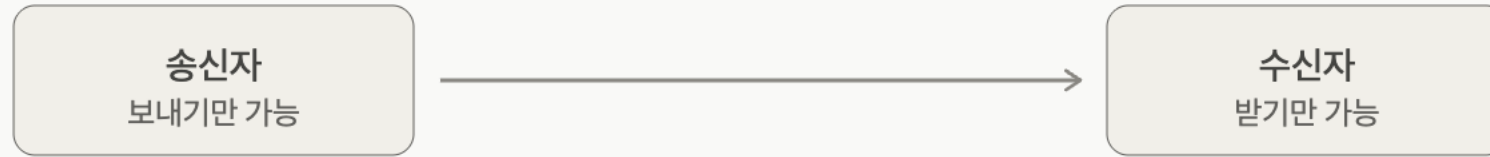


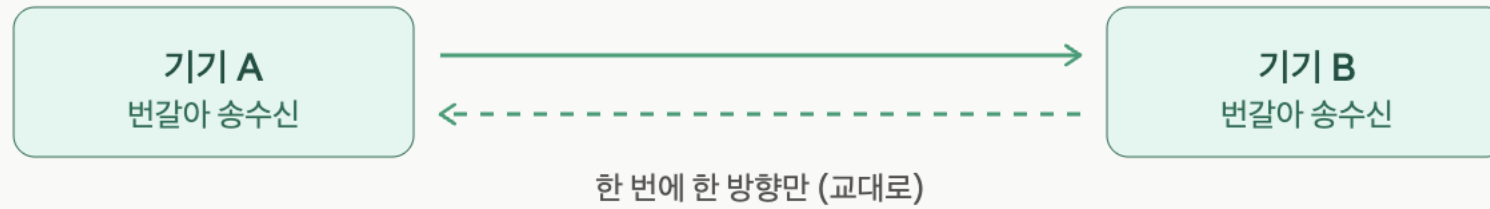
양방향통신 단방향통신

오승연

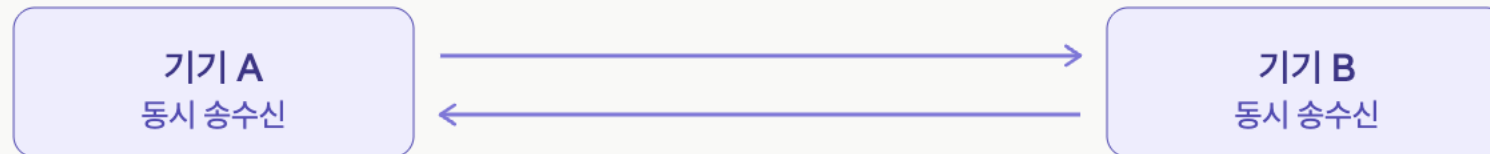
단방향 (Simplex)



반이중 (Half-Duplex)



전이중 (Full-Duplex)



구분	단방향	반이중	전이중
방향	한 방향만	양방향 (교대)	양방향 (동시)
동시 송수신	✕	✕	✓
채널 수	1	1 (공유)	2 (또는 분리 기술)
충돌 가능성	없음	있음	없음
대역폭 효율	전체를 한 방향에	방향당 절반 수준	방향당 전체
예시	방송, 키보드	무전기, 허브	전화, 스위치

1. HTTP 요청-응답 모델

- **동작 방식:** 클라이언트(브라우저)가 요청(Request)을 보내면 서버가 응답(Response)을 돌려주고, 연결이 끝납니다. 서버는 클라이언트가 묻기 전에는 절대 먼저 말을 걸 수 없습니다.

```
@GetMapping("/api/boards")
public List<BoardDto> getBoards() {
    return boardService.findAll(); // 요청이 와야만 실행됨
}
```

2. 폴링(Polling)

- **동작 방식:** 클라이언트가 일정 주기(예: 5초)마다 서버에 "새 데이터 있어?"라고 반복 요청합니다.

// 클라이언트: 5초마다 새 알림 확인

```
setInterval(async () => {  
    const res = await fetch('/api/notifications/new');  
    const data = await res.json();  
    if (data.length > 0) showNotification(data);  
}, 5000);
```

3. 롱 폴링(Long Polling)

- 동작 방식: 클라이언트가 요청을 보내면, 서버가 바로 응답하지 않고 새 데이터가 생길 때까지 연결을 열어둔 채 기다립니다. 데이터가 생기면 그때 응답하고, 클라이언트는 응답을 받자마자 즉시 다음 요청을 다시 겁니다.

```
@GetMapping("/api/notifications/poll")
public DeferredResult<List<Notification>> poll(@RequestParam Long userId) {
    // 30초 타임아웃, 그 안에 데이터가 생기면 즉시 응답
    DeferredResult<List<Notification>> result = new DeferredResult<>(30_000L);
    notificationService.register(userId, result);
    return result; // 여기서 리턴해도 응답은 아직 안 나감!
}

// 다른 스레드에서 새 알림 발생 시
public void onNewNotification(Long userId, Notification n) {
    DeferredResult<List<Notification>> waiting = registry.get(userId);
    if (waiting != null) {
        waiting.setResult(List.of(n)); // 이 순간 HTTP 응답이 나감
    }
}
```

4. SSE (Server-Sent Events)

- **동작 방식:** http유지. 라디오같다. 클라이언트가 **한 번** 연결하면, 서버가 그 연결을 닫지 않고 이벤트를 **계속 스트리밍**합니다. 방향은 **서버 → 클라이언트 단방향**입니다.

실제 사용 예:

- 주식/코인 시세 표시 (서버가 일방적으로 뿌려주기만 하면 됨)
- 알림 뱃지 갱신
- ChatGPT/Claude가 답변을 한 글자씩 스트리밍하는 것도 SSE 방식입니다
- 배치 작업 진행률 표시 (0%... 37%... 100%)

4. SSE (Server-Sent Events)

```
@GetMapping(value = "/api/stock/subscribe", produces = MediaType.TEXT_EVENT_STREAM)
public SseEmitter subscribe() {
    SseEmitter emitter = new SseEmitter(60 * 60 * 1000L); // 1시간
    emitterRegistry.add(emitter);
    return emitter;
}
```

// 시세 변동 시 모든 구독자에게 push

```
public void broadcast(StockPrice price) {
    for (SseEmitter emitter : emitterRegistry.getAll()) {
        emitter.send(SseEmitter.event()
            .name("price-update")
            .data(price));
    }
}
```

// 클라이언트 - 브라우저 내장 API, 매우 간단

```
const es = new EventSource('/api/stock/subscribe');
es.addEventListener('price-update', (e) => {
    updateUI(JSON.parse(e.data));
});
```

// 연결이 끊기면 브라우저가 자동 재연결해줌 (WebSocket엔 없는 장점!)

5. WebSocket

- **동작 방식:** 처음에 HTTP로 시작해서 "**프로토콜 업그레이드**" **핸드셰이크**를 거친 뒤, 그 TCP 연결을 유지하면서 **양쪽이 아무 때나 서로에게** 메시지를 보낼 수 있는 **전이중** 채널로 전환합니다.

실제 사용 예:

- 카카오톡 같은 **채팅** (상대 메시지가 즉시 도착해야 함)
- 실시간 협업 (구글 독스, 피그마 — 다른 사람 커서가 움직이는 것)
- 온라인 게임
- 실시간호가창 + 주문 (조회와 주문이 모두 실시간이어야 하는 경우)

5. WebSocket

오늘부터 웹소켓하지않을래



○○ 웹소켓으로
업그레이드해줌



5. WebSocket

1. 터미널 오픈해서 npm init 입력
2. npm install express ws 입력

3. 웹서버만드는 코드

```
JS server.js  X
JS server.js > ...
const express = require('express');
const app = express();

app.use("/", function(req, res){
  res.sendFile(__dirname + '/index.html');
});

app.listen(8080);

const WebSocket = require('ws');

const socket = new WebSocket.Server({
  port: 8081
});
```

이게 웹소켓 열기 끝임

5. WebSocket

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h4>채팅페이지입니다</h4>
  <button id="send" onclick="socket.send('ㅎㅇ')">메세지 보내기</button>

  <script>
    let socket = new WebSocket("ws://localhost:8081");
  </script>
</body>
</html>
```

웹소켓으로 유저 -> 서버 메세지 보내려면

웹소켓 열어달라고 서버에게 부탁하는 코드

```
socket.on('connection', (ws, req)=>{
  ws.on('message', (msg)=>{
    console.log('유저가 보낸거 : ' + msg);
    ws.send('꺼져');
  })
});
```

웹소켓으로 서버 -> 유저 메세지 보내려면

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

새로운 크로스 플랫폼 PowerShell 사용 <https://aka.ms/pscore6>

PS C:\Users\haeyu\Desktop\socket> node server.js

유저가 보낸거 : ㅎㅇ
유저가 보낸거 : ㅎㅇ
유저가 보낸거 : ㅎㅇ
유저가 보낸거 : ㅎㅇ

ws 대신 socket.io 라이브러리 쓰면

- 연결 끊기면 자동재접속 기능
- 웹소켓 접속자마다 자동 id 부여
- 모든 웹소켓유저에게 전체 메세지 전송가능
- 웹소켓방을 생성가능

5. WebSocket

상위 프로토콜 / 라이브러리

용어

설명

STOMP

WebSocket 위에 얹는 메시징 프로토콜. 목적지(destination), 구독 개념 제공. Spring 표준 조합

SockJS

WebSocket 미지원 환경에서 롱 폴링 등으로 자동 폴백해주는 라이브러리

Socket.IO

Node.js 진영의 대표 라이브러리. 자동 재연결, 룸(room), 폴백 내장 — WebSocket 자체가 아니라 그 위의 프레임워크

폴백 (Fallback)

WebSocket이 안 될 때 롱 폴링 등 대체 수단으로 자동 전환하는 것

서브프로토콜
(Subprotocol)

핸드셰이크에서 `Sec-WebSocket-Protocol` 헤더로 협상하는 상위 규약 (STOMP 등)

6. gRPC 양방향 스트리밍

- **동작 방식:** gRPC는 HTTP/2 기반 RPC 프레임워크인데, HTTP/2의 **멀티플렉싱 스트림** 덕분에 하나의 연결 위에서 클라이언트와 서버가 **각자 독립적인 속도로 동시에** 메시지 스트림을 주고받을 수 있습니다.

4가지 통신 패턴을 제공합니다:

패턴	방향	예시
Unary	요청 1 → 응답 1	일반 API 호출과 동일
Server streaming	요청 1 → 응답 N	대용량 조회 결과를 나눠 받기
Client streaming	요청 N → 응답 1	파일을 조각내서 업로드
Bidirectional streaming	요청 N ↔ 응답 N 동시	실시간 번역, 음성 인식

정 리

폴링 / 롱 폴링

- 페이스북 초기 채팅 — 롱 폴링으로 시작했던 대표 사례
- 카카오톡 — 순수 WebSocket이 아니라 자체 프로토콜 + 롱 폴링 계열 기법을 섞어 쓰는 것으로 알려져 있음
- 여전히 많은 사내 시스템, 관리자 대시보드가 단순 폴링 사용

SSE

- ChatGPT / Claude — 답변이 한 글자씩 흘러나오는 스트리밍이 SSE
- 토스 — 홈 화면 실시간 갱신 등에 SSE 활용 사례를 기술 블로그에 공개한 적 있음
- 각종 서비스의 알림 뱃지, 주식 앱의 시세 표시

WebSocket

- 슬랙(Slack) — 메시지 실시간 수신
- 디스코드(Discord) — 채팅 + 상태 표시(온라인/오프라인)
- 피그마(Figma) — 다른 사람 커서가 실시간으로 움직이는 것
- 업비트/바이낸스 — 실시간 호가창
- 구글 독스 — 동시 편집

gRPC (스트리밍 포함)

- 넷플릭스(Netflix) — 내부 마이크로서비스 간 통신
- 구글 — 애초에 구글이 내부에서 쓰던 걸 오픈소스화한 것 (유튜브, 구글 지도 등의 백엔드)
- 우버(Uber) — 차량 위치 스트리밍 등 서비스 간 통신
- 쿠버네티스(Kubernetes) 생태계 — etcd 등 인프라 도구들이 gRPC 기반

화면에서 직접 보는 실시간 기능(채팅, 커서, 호가창)은 대부분 WebSocket이나 SSE이고,
그 뒤에서 서버들끼리 주고받는 통신은 gRPC

기술	장점	단점
폴링	구현이 가장 단순, 어떤 환경에서도 동작, 서버 상태 관리 불필요	헛된 요청으로 서버/네트워크 낭비, 주기만큼 지연 발생 (실시간 아님)
롱 폴링	폴링보다 지연 적음, HTTP 그대로라 호환성 좋음	매 응답마다 재연결 → 헤더 오버헤드 반복, 서버가 대기 연결을 계속 붙들어야 함
SSE	구현 간단 (브라우저 내장 API), 자동 재연결 내장, HTTP 기반이라 인증·방화벽 문제 적음	단방향 (클라→서버는 별도 요청 필요), 텍스트만 전송 가능, 브라우저당 동시 연결 수 제한 (HTTP/1.1 기준 6개)
WebSocket	진짜 양방향 동시 통신, 헤더 오버헤드 거의 없어 빠름, 바이너리 지원	자동 재연결 없음 (직접 구현), 연결 유지 비용, 서버 여러 대면 브로커(Redis 등) 필요, 일부 프록시/방화벽과 충돌
gRPC 스트리밍	HTTP/2 기반 고성능, proto로 타입 안전 보장, 4가지 스트리밍 패턴 유연	브라우저 직접 지원 안 됨, 학습 곡선 높음, 디버깅 어려움 (바이너리라 눈으로 못 읽음), 사내 REST 문화와 이질적